1. Os serviços precisam se comunicar. Como fazer isso?

   A comunicação entre os container se dá através da criação de uma rede virtual compartilhada personalizada. Fazemos isso com o docker network.

2. Por que utilizar o nome do serviço é mais adequado do que utilizar diretamente o IP do container?

   Os endereços IP atribuídos aos containers são dinâmicos, o que significa que podem mudar a cada reinicialização. Utilizando o nome do serviço, a aplicação se utiliza do servidor DNS interno do Docker. Isso colabora para que, mesmo que o IP do container mude, os nomes continuem apontando para a instância certa.

3. Por que foi necessário utilizar --build?

   Por predefinição, o comando docker compose up reutiliza as imagens já construídas que ficam na cache local. O --build força a reconstrução da imagem da aplicação, invalidando camadas desatualizadas e garantindo que o novo código seja contido no container recriado.

4. Qual é a função do Dockerfile na aplicação?

   O Dockerfile descreve e define o passo a passo para a compilação da imagem Docker personalizada da aplicação.

5. Qual é a função do Redis nessa arquitetura?

   O Redis atua como camada de armazenamento de estado e base de dados em memória do sistema distribuído. A sua função é guardar e persistir a contagem de acessos de forma centralizada e desacoplada do servidor web.

6. Como o container Flask consegue encontrar o Redis sem conhecer seu endereço IP?

   Através do servidor DNS interno integrado do Docker. Quando os containers estão conectados a uma rede definida pelo pelo usuário, o Docker resolve o *hostname* correspondente ao nome do serviço ou do contentor diretamente para o endereço IP interno transitório que foi atribuído ao Redis naquela sub-rede.

7. Qual é a função do volume utilizado pelo Redis?

   A função do volume é garantir a persistência dos dados gravados pelo Redis além do ciclo de vida útil do container. Já que o sistema de ficheiros padrão de um container é volátil, e os dados seriam descartados se ele fosse removido.

8. Qual problema o Docker Compose resolve em relação à execução manual dos containers?

   O Docker Compose resolve a complexidade de orquestração, configuração manual e reprodutibilidade de aplicações com vários containers. O Docker Compose centraliza toda a infraestrutura de comandos em um ficheiro declarativo único (docker-compose.yml), permitindo iniciar, parar ou recriar todo o ecossistema com um único comando (docker compose up / docker compose down).